

COMP 418/518: Homework 4

[Weight of homework: 25% of the final grade]

Authors: Yicheng Jin yj67@rice.edu

Due on April 6, 2026
(released on March 23, 2026)

Instructions: For this homework, you should work in groups of 3–4 people (i.e., the groups that have been declared and accepted). Only one member of the group should submit on Gradescope. All group members must be included in the submission on the Gradescope system (otherwise no credit will be given). Also specify in your submission file all the members of your group (name and NetID) using the `\authors` macro. The submission must be typed in LaTeX using the provided template. The use of AI systems (including, but not limited to, ChatGPT, Gemini, Copilot, and Claude) is entirely prohibited. No discussion or collaboration is allowed outside of the group. Late submissions are not accepted.

You should provide *pseudocode* for all the algorithms that you describe.

Grading: We will give a 5% grade bonus to students taking COMP 418. The homework will be graded based on correctness, completeness, and the quality of the provided solutions.

Note: Please go over the provided Java source code, because it contains some definitions that are relevant for several questions of the homework. Unless specified otherwise, all the denotations that we consider in the problems are cumulative (i.e., they give the overall output history).

1 The Dataflow Model [110 points]

We will explore the dataflow model of computation. We consider dataflow graphs where each process (sometimes referred to as node, operator, actor) can be implemented using the following Java interface:

```
public interface Operator<A,B> {
    void start(Sink<B> sink);
    void next(A item, Sink<B> sink);
}

public interface Sink<A> {
    void next(A item);
}
```

See the uploaded code for a similar programming model.

We assume that every execution of `start` and `next` terminates and thus results in the generation of a *finite* number of output items. Under this assumption, every operator can be described by a *monotone* function $f : A^* \rightarrow B^*$ (cumulative denotation), where A^* is the set of all finite words (strings) over A . See several lectures that discuss this concept of describing operators (streaming algorithms) using functions.

1.1 Semantic Characterization of Statelessness [10 points]

You are asked to give a semantic characterization of the *stateless* operators, that is, those operators that can be implemented without maintaining any state across steps (examples of such operators: `id`, `filter`, and `map`). The operators `fold` and `scan`, on the other hand, are stateful. More specifically, you want to characterize the class of (computable) monotone functions $f : A^* \rightarrow B^*$ that admit a stateless implementation. Formulate precisely a property $P(f)$ so that Theorem ?? below holds.

Definition of property $P(f)$: $P(f)$: There exists a function $h : A \rightarrow B^*$ such that for every $s \in A^*$ and every $a \in A$:

$$f(s \cdot a) = f(s) \cdot h(a).$$

Equivalently, the *incremental output* produced upon receiving input a depends only on a and not on the history s .

Theorem 1. Let $f : A^* \rightarrow B^*$ be a computable monotone function. The function f is the cumulative denotation of a stateless operator if and only if $P(f)$.

Proof. ($\Rightarrow$) Suppose f is the cumulative denotation of a stateless operator. Then `start(sink)` produces some fixed output $w_0 = f(\varepsilon) \in B^*$, and `next(a, sink)` produces output that depends only on a (no state across steps), say $h(a) \in B^*$. For any input $s = a_1 a_2 \cdots a_n$:

$$f(a_1 \cdots a_n) = w_0 \cdot h(a_1) \cdot h(a_2) \cdots h(a_n).$$

In particular, $f(s \cdot a) = f(s) \cdot h(a)$ for all $s \in A^*$ and $a \in A$. So $P(f)$ holds.

($\Leftarrow$) Suppose $P(f)$ holds with witness function $h : A \rightarrow B^*$. We construct a stateless operator:

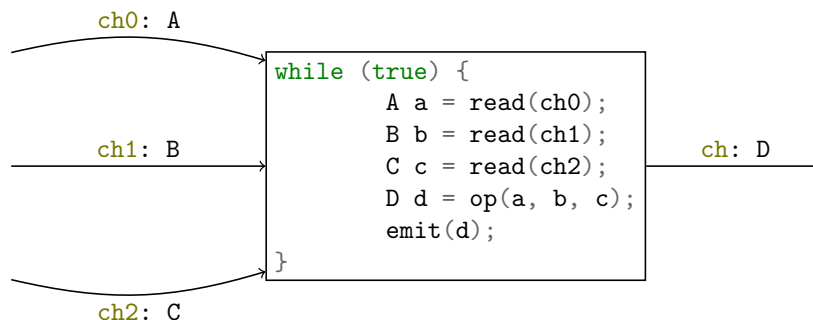
- `start(sink)`: emit the symbols of $f(\varepsilon)$ to `sink`.

- `next(a, sink)`: emit the symbols of $h(a)$ to `sink`. No state is needed because $h(a)$ depends only on a .

This operator uses no mutable state across calls to `next`. Its cumulative denotation on input $a_1 \cdots a_n$ is $f(\varepsilon) \cdot h(a_1) \cdots h(a_n)$, which equals $f(a_1 \cdots a_n)$ by induction using $f(s \cdot a) = f(s) \cdot h(a)$. Since h is computable (it can be extracted from f , which is computable), the implementation is valid. $\square$

1.2 Translations Between Programming Models [25 points]

We have used extensively a notation to describe dataflow processes using `read(channel)` to read from an input channel and `emit(channel)` to write to an output channel. This programming model was introduced by Kahn in [?] and was further elaborated in [?]. If you want to learn more about these seminal works you can read the survey [?] and the notes [?] (see reading list on Canvas). Consider the following Kahn process:



Describe the same computation by implementing the `Zip3` operator of type `Operator<Or3<A,B,C>,D>`. The three input channels of the Kahn process are encoded using the disjoint union (coproduct) `Or3<A,B,C>`, which is a straightforward extension of `Or<A,B>` (see the uploaded code). Fill pseudocode in Figure ???. You do not have to write actual Java code for this.

Consider now the operator `Count2` of type `Operator<Or<A,A>,Nat>` that is shown in Figure ???. The input can be thought of as the interleaving of two input channels. Translate `Count2` into a Kahn process with two input channels (if possible), or prove that it is not possible to do so.

Operator to Kahn process translation: It is **not possible** to translate `Count2` into a Kahn process with two input channels.

Proof. A Kahn process is *deterministic*: at each step, it must commit to reading from a specific channel. Its input/output behavior (as a function of the two input streams) is completely determined by the contents of the two channels, regardless of the timing or interleaving of arrivals.

Consider two scenarios with the same pair of input streams $s = (a_1)$ on `ch0` and $t = (a_2)$ on `ch1`. A Kahn process must choose which channel to read first. Suppose it reads `ch0` first:

- It reads a_1 from `ch0`, emits 1, then reads a_2 from `ch1`, emits 2. So the output is $(1, 2)$.

Now consider $s = \varepsilon$ (empty) on `ch0` and $t = (a_2)$ on `ch1`. The Kahn process tries to read `ch0` first, but `ch0` is empty, so it *blocks forever*. It never reads from `ch1` and produces no output (ε). But `Count2` should output (1) since one total item was received. Contradiction.

A symmetric argument applies if the process reads `ch1` first. More generally, any deterministic reading schedule will block when the channel it tries to read next has no data, even though the other channel may have items. `Count2` requires a *fair merge* (reacting to whichever channel has data), which is inherently *non-deterministic* and cannot be expressed as a Kahn process. $\square$

```

public class Zip3<A, B, C, D> implements Operator<Or3<A,B,C>, D> {

    private final Func3<A,B,C,D> op;
    private Queue<A> bufA;
    private Queue<B> bufB;
    private Queue<C> bufC;

    public Zip3(Func3<A,B,C,D> op) {
        this.op = op;
        this.bufA = new Queue<>();
        this.bufB = new Queue<>();
        this.bufC = new Queue<>();
    }

    @Override
    public void start(Sink<D> sink) {
        // do nothing
    }

    @Override
    public void next(Or3<A,B,C> item, Sink<D> sink) {
        // item.match(
        //     a -> bufA.enqueue(a),
        //     b -> bufB.enqueue(b),
        //     c -> bufC.enqueue(c)
        // );

        item.map(
            a -> bufA.enqueue(a),
            b -> bufB.enqueue(b),
            c -> bufC.enqueue(c),
        );

        while (!bufA.isEmpty() && !bufB.isEmpty() && !bufC.isEmpty()) {
            A a = bufA.dequeue();
            B b = bufB.dequeue();
            C c = bufC.dequeue();
            sink.next(op.apply(a, b, c));
        }
    }
}

```

Figure 1: The Zip3 operator.

```
public class Count2 implements Operator<Or<A,A>, Nat> {  
  
    private long count = 0;  
  
    @Override  
    public void start(Sink<Nat> sink) { /* do nothing */ }  
  
    @Override  
    public void next(Or<A,A> item, Sink<Nat> sink) {  
        A a = item.map(x -> x, x -> x);  
        count += 1;  
        sink.next(count);  
    }  
}
```

Figure 2: The Count2 operator.

1.3 Denotations [10 points]

Describe the denotation of **Zip3** as a function $f : A^* \times B^* \times C^* \rightarrow D^*$ and the denotation of **Count2** as a function $g : A^* \times A^* \rightarrow \mathbb{N}^*$, where $\mathbb{N}$ is the set of the natural numbers (nonnegative integers).

Solution: Denotation of Zip3. Let $s = a_1 \cdots a_p \in A^*$, $t = b_1 \cdots b_q \in B^*$, $u = c_1 \cdots c_r \in C^*$, and let $n = \min(p, q, r)$. Then:

$$f(s, t, u) = op(a_1, b_1, c_1) \cdot op(a_2, b_2, c_2) \cdots op(a_n, b_n, c_n).$$

That is, f zips the three streams element-wise and applies op , producing output only up to the length of the shortest input stream.

Denotation of Count2. Let $s \in A^*$ and $t \in A^*$ with $|s| = p$ and $|t| = q$. Then:

$$g(s, t) = 1 \cdot 2 \cdots (p + q).$$

That is, $g(s, t)$ is the sequence $(1, 2, \dots, p + q)$. The output is simply the running count of the total number of items received from both channels, regardless of the interleaving order or the values of the items.

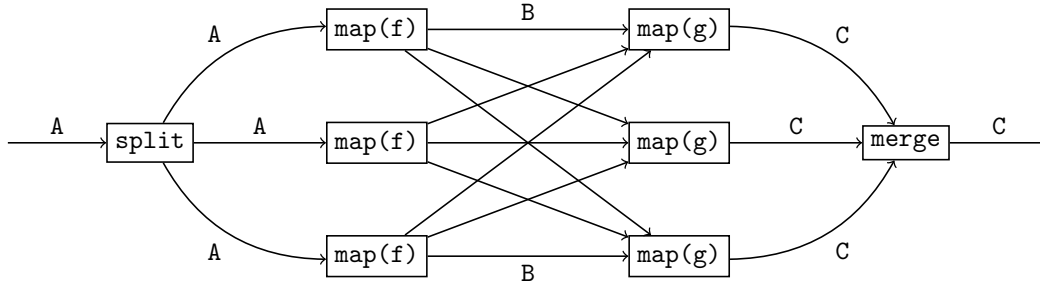
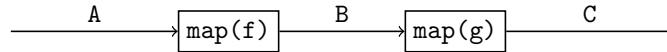


Figure 3: Extracting data parallelism from a simple pipeline of stateless operators.

1.4 Data Parallelism [25 points]

In applications that require the processing of massive amounts of data (e.g., data generated by a large number of high-resolution sensors) it is often desirable to exploit *data parallelism*. This is what the distributed data processing systems MapReduce, Spark, Storm and others do. As a simple example of this, consider a pipeline consisting of two `map` operators:



We wish to parallelize this pipeline by creating several instances of `map(f)` and `map(g)` that compute independently. This is a meaningful transformation when `f` and `g` are computationally expensive. This transformation gives rise to dataflow graph of Figure ??.

Describe each of the nodes (processes) of the dataflow graph of Figure ?. You can assume that every communication channel is reliable (no messages are lost) and behaves like a FIFO buffer. Make sure that the behavior of the graph of Figure ? (in terms of the input/output function) is exactly the same as the behavior of the original pipeline. Does this require making any noteworthy changes to the graph? You may have to introduce new notation for writing pseudocode that describes the nodes. Try to make your notation intuitive and explain the aspects that are not obvious. You do not have to restrict yourself to Kahn processes for the code that runs on each node.

Solution: We use the notation `read(ch)` for blocking read from a channel and `emit(ch, v)` for writing value `v` to a channel. We denote the three parallel lanes as lane 0, 1, 2. Each item is tagged with a *sequence number* to allow the merge node to reconstruct the original order.

Noteworthy change to the graph: The diagram shows a *fully connected* bipartite graph between the `map(f)` and `map(g)` stages (9 edges). However, for correctness we only need each `map(f)k` to send to the corresponding `map(g)k` on the same lane (3 edges). The cross-lane connections are unnecessary and would cause incorrect duplicate processing. So the graph should be simplified to have only the 3 direct lane connections between the two stages.

Split node:

```
// Distributes items round-robin across 3 lanes, tagging with sequence number
seq = 0;
while (true) {
    A a = read(input);
```



```

    lane = seq % 3;
    emit(out[lane], (a, seq));    // send tagged item to lane
    seq++;
}

```

Map(f)_k node (for lane $k \in \{0, 1, 2\}$):

```

// Applies f to each item, passes along the sequence number
while (true) {
    (A a, int seq) = read(in_k);
    B b = f(a);
    emit(out_k, (b, seq));
}

```

Map(g)_k node (for lane $k \in \{0, 1, 2\}$):

```

// Applies g to each item, passes along the sequence number
while (true) {
    (B b, int seq) = read(in_k);
    C c = g(b);
    emit(out_k, (c, seq));
}

```

Merge node: Fangzhou here !

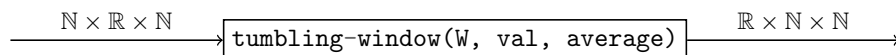
Correctness argument: Since `split` distributes items in round-robin order (item i goes to lane $i \bmod 3$), the merge node knows exactly which lane holds the next item in sequence. It reads from lane $nextSeq \bmod 3$, which is deterministic and requires no buffering. The overall input/output function is: for input $a_1 a_2 \cdots a_n$, the output is $g(f(a_1)) \cdot g(f(a_2)) \cdots g(f(a_n))$, which is identical to the original pipeline $\text{map}(f) \gg \text{map}(g)$.

1.5 Disorder and Watermarks [40 points]

Consider an IoT application that requires collecting data from a large and physically distributed collection of smart sensors. All sensor streams are sent to a centralized location (“server”) for processing. Because of the physical distribution, the data can arrive disordered. The server sees a single input stream of data items of the form $(sensorId, val, ts) : \mathbb{N} \times \mathbb{R} \times \mathbb{N}$, where $sensorId : \mathbb{N}$ is a unique identifier for a sensor, $val : \mathbb{R}$ is the sensor measurement, and $ts : \mathbb{N}$ is a timestamp that measures time in msec (milliseconds). The data items in the input stream may be *disordered* with respect to the timestamp. So, we *cannot* expect in general that the data will appear in non-decreasing timestamp order.

We define the *watermark* of the input stream to be the maximum timestamp seen so far¹. We make a *bounded lateness* assumption, namely that each data item should be at most $T \geq 0$ time units behind the watermark. That is, if $(sensorId, val, ts)$ is the current data item and t is the current watermark, then it should hold that $t - ts \leq T$.

We want to compute the *tumbling window average* of the measurement values. Suppose that $W > 0$ is the length of the window. For each time interval $[0, W), [W, 2W), [2W, 3W), \dots$ we want to output the average value of all measurements that belong to the interval according to their timestamp. The output should be of the form $(avg, wndStartTs, wndEndTs) : \mathbb{R} \times \mathbb{N} \times \mathbb{N}$, where $avg : \mathbb{R}$ is the average measurement of the window, $\mathbb{N}$ is the start time of the window, and $\mathbb{N}$ is the end time of the window. E.g., if y_0 is the average of the window $[0, W)$, then the output should be $(y_0, 0, W - 1)$.



You should emit the output for a window as early as possible, i.e., as soon as it becomes apparent that no other measurement of the window will be seen in the future (recall the bounded lateness assumption).

Provide detailed streaming pseudocode for the most efficient implementation of this tumbling window operator that you can think of. Discuss the memory and time-per-item complexity of your implementation.

Solution:

Key idea. We maintain per-window aggregates (sum and count). The watermark is the maximum timestamp seen so far. A window $[kW, (k+1)W)$ is emitted as soon as the watermark guarantees no future item can belong to it: this happens when $watermark \geq (k+1)W + T$, because the bounded lateness assumption ensures every future item has $ts \geq watermark - T \geq (k+1)W$, which falls outside window k .

Pseudocode:

```
// State
watermark = 0;
windows = new HashMap<Int, (Double sum, Int count)>();
nextWindowToEmit = 0; // index of the earliest un-emitted window
```

¹Similar notions of watermarks are considered in many data stream processing systems. For example, see <https://learn.microsoft.com/en-us/azure/stream-analytics/stream-analytics-time-handling> and <https://www.databricks.com/blog/feature-deep-dive-watermarking-apache-spark-structured-streaming>.

```

// Operator interface
void start(Sink<(Real, Nat, Nat)> sink) {
    // do nothing
}

void next((Nat sensorId, Real val, Nat ts), Sink<(Real, Nat, Nat)> sink) {
    // 1. Update watermark
    watermark = max(watermark, ts);

    // 2. Assign item to its window and update aggregate
    int k = ts / W;    // integer division: window index
    if (windows.containsKey(k)) {
        windows[k].sum += val;
        windows[k].count += 1;
    } else {
        windows[k] = (val, 1);
    }

    // need to be fixed, store old watermark, the complexity of per item may not be O(1),

    // 3. Emit all windows that are now complete
    // Window k is complete when watermark >= (k+1)*W + T
    while (nextWindowToEmit * W + W + T <= watermark) {
        int k = nextWindowToEmit;
        if (windows.containsKey(k)) {
            Real avg = windows[k].sum / windows[k].count;
            sink.next((avg, k * W, (k+1) * W - 1));
            windows.remove(k);
        }
        // If no items fell in this window, skip it (no output)
        nextWindowToEmit++;
    }
}

```

Complexity analysis.

- **Time per item:** Updating the watermark and inserting into the window map are both $O(1)$ (using a hash map). The `while` loop emits completed windows. Each window is emitted at most once across the entire stream, so the total cost of all emissions is proportional to the number of windows emitted. Amortized over all items, the cost per item is $O(1)$.
- **Memory:** At any point, the active (un-emitted) windows are those with index $k \geq \text{nextWindowToEmit}$ that have received at least one item. Since all items satisfy $ts \geq \text{watermark} - T$, the timestamps of buffered items span a range of at most $T + W$. Therefore, the number of active windows is at most $\lfloor T/W \rfloor + 2$. Each window stores a sum and a count, so the total memory is $O(T/W + 1)$.

2 Denotational Semantics of Kahn Process Networks [40 points]

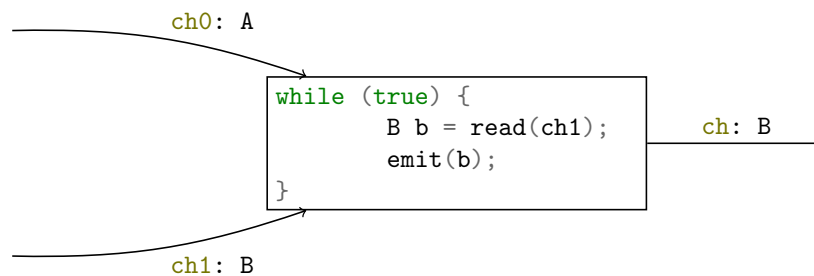
Lemma 2. Let A and B be ω -CPOs. Then, $A \times B$ is an ω -CPO.

Proof. **Fill me in.** □

Lemma 3. Let A , B and C be ω -CPOs. Suppose that $f : A \rightarrow B$ and $g : B \rightarrow C$ are ω -continuous. Then, the serial composition $f \gg g : A \rightarrow C$ is ω -continuous. Recall from the lectures that $(f \gg g)(a) = g(f(a))$ for every $a \in A$.

Proof. **Fill me in.** □

We will consider now the *semantics of feedback* using an interesting example. Consider the Kahn process shown below:



The (cumulative) input/output behavior of this process can be described by an ω -continuous function $f : A^\infty \times B^\infty \rightarrow B^\infty$. Describe the function f and the function $\text{loop}(f) : A^\infty \rightarrow B^\infty$ (loop is feedback composition, as defined in the lectures).

Solution: **Fill me in.**

3 ToyDSL [100 points]

Download the Java project from Canvas. Carefully read all the provided code in order to understand the programming model that we consider. Implement the ToyDSL domain-specific language using the included instructions. Your implementation should pass all included unit tests. If there is anything that you would like to clarify, please do so below.

Solution

Fill me in.

References

- [1] Gilles Kahn. The semantics of a simple language for parallel programming. *Information Processing*, 74:471–475, 1974.
- [2] Gilles Kahn and David B. MacQueen. Coroutines and networks of parallel processes. *Information Processing*, 77:993–998, 1977.
- [3] Edward A. Lee. *Concurrent Models of Computation: An Actor-Oriented Approach*. 2011.
- [4] David B. MacQueen. Kahn networks at the dawn of functional programming. In Yves Bertot, Gérard Huet, Jean-Jacques Lévy, and Gordon Plotkin, editors, *From Semantics to Computer Science: Essays in Honour of Gilles Kahn*, pages 95–137. Cambridge University Press, 2009.